FloodGuard: Cloud Service Integration and Monitoring Through Serverless and Microservices Architecture

Ashok Bhattrai (NP0698[##]), Samir Pokhrel (NP0698[##]),

Anjali Misra (NP0698[##]), and Sangita Tamang (NP0698[##])

Asia Pacific University of Technology and Innovation

CT071-3-3-DDAC: Designing and Developing Cloud Applications

[LECTURER NAME]

[SUBMISSION DATE]

Group G[GROUP NUMBER], Problem Background 4: Flood Early Warning

Table of Contents

Introduction

Flooding remains one of the most destructive recurring natural hazards, and the gap between a rising river and a warned population is frequently measured in minutes rather than hours. FloodGuard addresses Problem Background 4 by providing a cloud-native early warning and emergency response platform for residents, volunteers, and municipal authorities. Task 1 delivered a working server-based implementation of that platform. Task 2, documented here, extends it with a serverless, event-driven tier and integrates the monitoring instrumentation required to evaluate the result.

The objective of this report is threefold. First, it summarises the system inherited from Task 1 and states precisely how the architecture was expanded. Second, it documents the implementation of the serverless components and the microservices that consume Amazon Simple Storage Service (S3), Amazon Simple Notification Service (SNS), and Amazon Simple Queue Service (SQS). Third, it presents monitoring evidence gathered from Amazon CloudWatch and AWS X-Ray and discusses what that evidence reveals about the performance of the expanded system. All resources described were deployed to a single AWS account in the us-east-1 region and were verified live before this report was written.

Task 1 Existing System Summary

Task 1 produced a two-tier server-based deployment. A Next.js frontend and a NestJS application server were each hosted on AWS Elastic Beanstalk, backed by a single Amazon Relational Database Service (RDS) instance running PostgreSQL 16. Three separate Amazon CloudFront distributions fronted the deployment, one for the web tier and others for the application programming interface (API).

All background processing ran inside the application server. Weather polling, flood risk scoring, and alert creation were executed by an in-process scheduled task using the @nestjs/schedule package. This design was functional but structurally constrained in three respects. The background work competed with user requests for the same compute resources; the scheduled task could not scale independently of the web workload; and, as Section *Changes in the Architecture* explains, the scheduler would have produced duplicate alerts had the environment ever scaled beyond a single instance. Table 1 summarises the inherited components.

Table 1
Components Inherited From the Task 1 Server-Based Deployment

Tier	Service	Configuration	Responsibility
Presentation	Elastic Beanstalk	Next.js 16, single instance	Server-rendered web interface
Application	Elastic Beanstalk	NestJS 11, load balanced	Business logic and REST API
Data	Amazon RDS	PostgreSQL 16.10, db.t3.micro	Relational persistence
Delivery	Amazon CloudFront	Three distributions	Transport security and caching
Scheduling	@nestjs/schedule	In-process cron, 10 min	Weather polling and alerting

Note. The scheduling row represents the component wholly replaced in Task 2. All other rows were retained and extended.

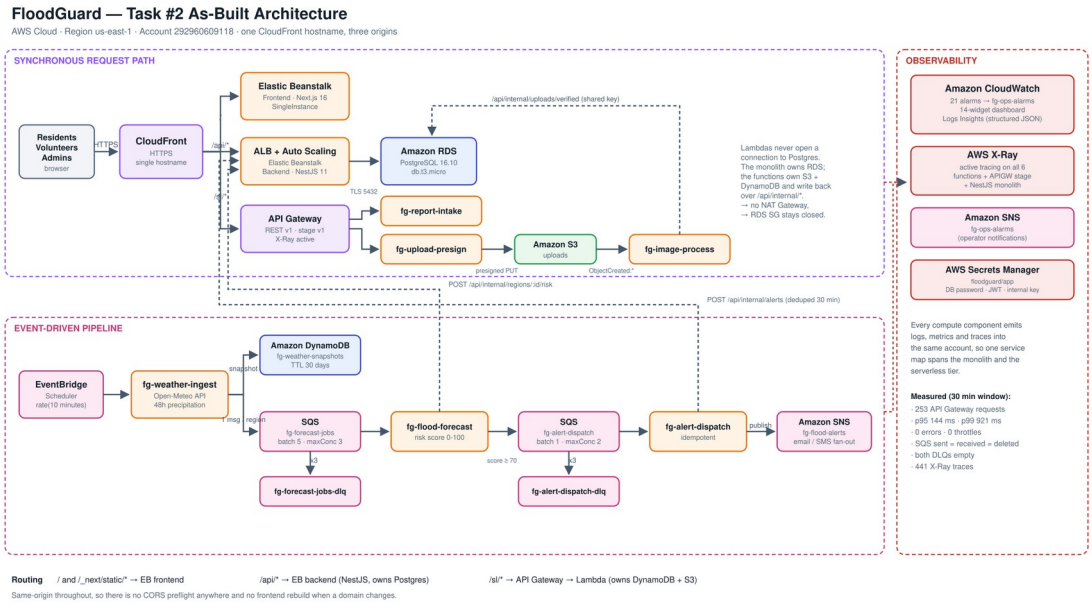
Proposed Cloud Architecture

The proposed architecture for Task 2 introduced a serverless tier alongside the retained server tier, on the principle that workloads with different scaling characteristics should not share a compute boundary. Scheduled ingestion, risk scoring, and notification fan-out are episodic and bursty, whereas the web and API tiers serve steady interactive traffic. Separating them permits each to scale on its own signal, a property Newman (2021) identifies as the primary operational benefit of decomposition.

The proposed design consolidated the three CloudFront distributions into a single distribution with path-based routing to three origins. It placed Amazon API Gateway in front of a set of AWS Lambda functions rather than in front of the existing application server, since only the former arrangement makes the functions independently addressable services. It introduced Amazon DynamoDB for append-only weather time-series data, SQS for decoupling the processing stages, and SNS for alert fan-out. Figure 1 presents the resulting architecture as deployed.

Figure 1

FloodGuard Task 2 As-Built Cloud Architecture



Note. The figure was generated from the deployed resource inventory rather than drawn from the design proposal, so every component shown corresponds to a provisioned resource. Solid arrows denote synchronous request flow; dashed arrows denote authenticated service-to-service calls.

Changes in the Architecture

Reviewing the proposed design against the Task 1 codebase before implementation identified several elements that would not have worked as originally drawn. Each was corrected,

and the corrections constitute the substantive architectural change between the two tasks. Table 2 records them.

Table 2

Architectural Changes Between Task 1 and Task 2

Element	Issue identified	Resolution
API Gateway placement	Positioned in front of Elastic Beanstalk, adding a network hop without adding capability. No request would reach a function, so no part of the system became serverless.	API Gateway placed in front of the Lambda functions.
API Gateway protocol	HTTP APIs do not emit AWS X-Ray trace segments, and X-Ray evidence is a requirement of this task.	REST API (v1) selected, which emits trace segments.
Trigger chain	EventBridge was drawn as publishing directly to SQS, but a scheduled tick carries no per-item work to distribute.	Chain reordered to EventBridge, then Lambda, then SQS, then Lambda.
Queue topology	Queue and dead-letter queue were drawn as one component, concealing the redrive mechanism.	Four queues provisioned: two work queues and two dead-letter queues.
S3 integration	S3 was drawn as a passive store with no event wiring.	Object-created notification invokes an image verification function.
DynamoDB purpose	Intended to hold live risk levels already authoritative in PostgreSQL, creating a dual write with no owner.	Repurposed for append-only weather time-series with a 30-day expiry.
Database exposure	Multi-availability-zone deployment in private subnets exceeded the available budget and required a network address translation gateway.	Single availability zone retained, with a two-zone subnet group so promotion remains a single API call.

Note. Each resolution was implemented and verified rather than proposed. The protocol decision in row 2 is discussed further in Section *API Gateway REST API*.

Two latent defects in the inherited system were also corrected. The first concerned duplicate alerting. Because @nestjs/schedule executes its cron on every running instance, and

because the Task 1 configuration constrained only instance type and not the minimum and maximum instance count, the first autoscaling event would have caused every flood alert to be delivered twice to real subscribers. Relocating the trigger to Amazon EventBridge Scheduler resolves this, since the scheduler fires once irrespective of fleet size, and the in-process cron is now disabled by a configuration flag. The second defect concerned credential handling: the Task 1 deployment configuration contained the live database password and token signing secret, committing both permanently to version control history. Credentials are now generated into AWS Secrets Manager and injected at deployment time.

Microservices

Six Lambda functions were implemented. Each is deployed with its own least privilege execution role, an arrangement that matters because the functions do not carry equivalent trust. A single shared role would grant the unauthenticated public intake endpoint permission to publish alerts to every subscriber, which is a privilege escalation path rather than a convenience.

Weather Ingestion Function

The fg-weather-ingest function is invoked on a ten-minute schedule by EventBridge. It retrieves 48-hour hourly precipitation forecasts from the Open-Meteo API (Open-Meteo, 2025) for each monitored region, writes a snapshot to DynamoDB, and publishes one message per region to the forecast work queue. The function performs input and output only. Risk scoring is deliberately excluded so that a slow change to the scoring algorithm cannot stall ingestion.

Flood Forecast Function

The fg-flood-forecast function consumes the forecast queue in batches of five and computes a composite risk score from zero to one hundred, combining weather contribution, sensor readings, and geographic exposure. Scores at or above the alert threshold are published to

the alert dispatch queue. The scoring logic was ported from the Task 1 application server without modification, which keeps the two implementations behaviourally identical.

Report Intake Function

The fg-report-intake function serves unauthenticated public flood reports received through API Gateway. It is the only write path in the system that requires no credential, and its execution role is correspondingly narrow.

Upload Presign Function

The fg-upload-presign function issues time-limited presigned upload URLs for S3, permitting a browser to transmit a photograph directly to object storage without the image bytes traversing the application server.

Image Processing Function

The fg-image-process function is invoked by an S3 object-created notification. A presigned URL constrains the declared content type but S3 does not verify that the uploaded bytes match that declaration, so this function performs the server-side check: it reads the file signature, tags the object as verified or rejected, and reports the verdict to the application server.

Alert Dispatch Function

The fg-alert-dispatch function consumes the alert queue one message at a time, creates the alert record, and publishes to the SNS fan-out topic. Because SQS provides at-least-once delivery, the function is idempotent: a matching active alert within a thirty-minute window is suppressed rather than duplicated.

Benefits of the Expansion

The expansion delivers four measurable benefits. Independent scalability is the first: each processing stage now responds to its own signal, with ingestion scaling on schedule, scoring on

queue depth, and dispatch on triggered alert volume, rather than by adding whole application instances. Second, the queue boundary confers deployment flexibility, since the scoring function can be replaced while messages accumulate and drain without request loss. Third, resilience improves through explicit dead-letter queues and partial batch failure reporting, which prevent a single malformed message from forcing redelivery of an entire batch. Fourth, cost efficiency improves because the functions consume compute only while executing, consistent with the economic model described by Adzic and Chatley (2017).

A further architectural benefit concerns data ownership. The Lambda functions never open a connection to PostgreSQL. The application server owns the relational database, the serverless tier owns DynamoDB and S3, and the two communicate over SQS and an authenticated internal HTTP interface. This preserves a single writer per data store, avoiding the dual-write consistency problem Richardson (2018) identifies as a common decomposition failure. It also keeps the functions outside the virtual private cloud, which avoids the recurring cost of a network address translation gateway and allows the database security group to remain closed.

System Implementation

Every resource described in this section was provisioned through the AWS Command Line Interface (CLI) rather than the management console, so that the deployment is reproducible and auditable. The commands are organised into numbered, idempotent step scripts under the project `infra/steps` directory; re-running any step converges on the desired state instead of creating duplicate resources. Where a provisioning command and a verification command are both given below, the verification command is read-only and may be run at any time.

S3 Storage Buckets

Create S3 Buckets

Two buckets were created: one receiving resident photograph uploads and one holding deployment artefacts. Bucket names embed the account identifier because the S3 namespace is global. Server-side encryption and versioning were enabled at creation, and a cross-origin resource sharing policy was applied to the uploads bucket so that browsers may complete a presigned upload directly.

Figure 2

CLI Resource Provisioning and Verification of the Amazon S3 Buckets

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the S3 buckets.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 22
Verification Command: aws s3api get-public-access-block --bucket floodguard-uploads-292960609118
Expected Evidence: Both bucket names reported, the AES256 encryption rule applied, and all four public access block settings true.
Placement: Immediately after the paragraph above.

Block Public Access

Neither bucket is publicly readable. Photographs are written through presigned URLs and read through CloudFront, so all four public access block settings were enabled. This is the control that prevents an accidental object-level access control list from exposing resident-submitted imagery.

DynamoDB Table

Create Weather Snapshot Table

A single DynamoDB table, fg-weather-snapshots, stores the forecast retrieved for each region at each ingestion cycle. The partition key is the region identifier and the sort key is the capture timestamp, which makes the most recent snapshots for a region a single efficient query. On-demand capacity was selected because ingestion traffic is periodic rather than sustained.

Figure 3

CLI Resource Provisioning and Verification of the Amazon DynamoDB Table

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the DynamoDB table.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 40
Verification Command: aws dynamodb describe-table --table-name fg-weather-snapshots --query 'Table.{Status:TableStatus,Keys:KeySchema,Billing:BillingModeSummary.BillingMode}'
Expected Evidence: Output shows TableStatus as ACTIVE, the regionId hash key and timestamp range key, and PAY_PER_REQUEST billing.
Placement: Immediately after the paragraph above.

Enable Time to Live Expiry

Weather snapshots have diminishing analytical value and unbounded growth would accrue cost indefinitely. A time-to-live attribute expires items after thirty days, which DynamoDB removes without consuming write capacity.

Verify Deployed Table

Confirming that the table contains data proves the ingestion pipeline is operating rather than merely provisioned. At the time of measurement the table held 3,212 items occupying approximately 782 kilobytes.

SNS Flood Alert Topics

Create Flood Alert Topic

The fg-flood-alerts topic performs public fan-out to residents by electronic mail and short message service. Subscriptions are created per subscriber, which allows SNS rather than application code to manage delivery retries.

Figure 4

CLI Resource Provisioning and Verification of the Amazon SNS Topics

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the SNS topics.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 42
Verification Command: aws sns list-topics --query "Topics[?contains(TopicArn,'fg-')]"
Expected Evidence: Both the fg-flood-alerts and fg-ops-alarms topic ARNs.
Placement: Immediately after the paragraph above.

Create Operations Alarm Topic

A second topic, fg-ops-alarms, carries CloudWatch alarm notifications to the operations team. The topics are deliberately separate: an operational alarm storm must never reach residents, and a public alert must never be mistaken for an infrastructure notification.

Secrets Manager Authentication

Create Application Secret

Authentication in FloodGuard operates at two levels. Human users authenticate to the application server with signed tokens, and the Lambda functions authenticate to the internal service interface with a shared key. Both the token signing secret and the internal key are generated at deployment time into a single Secrets Manager secret, floodguard/app, together with the database password. No credential value appears in source control or in terminal output.

Figure 5

CLI Resource Provisioning and Verification of the Application Secret

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the application secret.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 20
Verification Command: aws secretsmanager describe-secret --secret-id floodguard/app --query '{Name:Name,Changed:LastChangedDate}'
Expected Evidence: Output shows the secret name and last-changed date. Do not run get-secret-value and do not capture any secret material.
Placement: Immediately after the paragraph above.

Inject Secrets Into Runtime

Secret values are read at deployment time and applied as environment properties on the Elastic Beanstalk environments and as environment variables on the Lambda functions. The functions therefore receive the internal key without any developer handling it directly.

IAM Roles and Lambda Permissions

Create Lambda Trust Policy File

Each execution role requires a trust policy naming the Lambda service as the permitted principal. The policy document is written to a file so that role creation remains scriptable and reviewable.

Create Lambda Role

One role was created per function, following the principle of least privilege. Six roles therefore exist, each named after the function it serves.

Figure 6

CLI Resource Provisioning and Verification of the Lambda Execution Roles

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the six execution roles.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 43
Verification Command: aws iam list-roles --query "Roles[?starts_with(RoleName,'fg-')].RoleName" --output table
Expected Evidence: All six roles, one per function.
Placement: Immediately after the paragraph above.

Attach CloudWatch Logging Policy

The AWS managed basic execution role policy grants each function permission to create its log group and write log events. Without it a function executes but emits no diagnostic output, which would make the monitoring requirement unachievable.

Create Lambda App Policy File

Beyond logging, each function needs a narrowly scoped policy for the resources it actually touches. The ingestion function requires DynamoDB write and queue send permissions; the dispatch function requires queue receive and topic publish permissions; the presign function requires only object put permission on a single bucket prefix.

Create Custom App Policy

The per-function policy is applied as an inline role policy, which binds its lifecycle to the role and prevents an orphaned managed policy from persisting after teardown.

Attach Custom App Policy to Role

Because Identity and Access Management is eventually consistent, a function created immediately after its role can fail to assume it. The provisioning steps therefore retry role resolution before creating the function.

Verify Attached Role Policies

A final consolidated check across all six roles confirms that no role acquired a broader permission than intended.

Serverless Backend Folder Structure

The function source is organised one directory per function beneath infra/lambdas, with a shared directory copied into every deployment package at build time. This layout keeps each function independently deployable while avoiding duplicated helper code.

Figure 7
Directory Layout of the Serverless Backend
SCREENSHOT INSTRUCTION: Capture the function source directory tree.
Capture: Terminal, project root.
Command: find infra/lambdas -type f | sort
Expected Evidence: Six function directories plus _shared, each with an index.mjs handler.
Placement: Immediately after the paragraph above.

Weather Ingestion Lambda Folder

The ingestion handler batches its queue writes, because the SQS batch send operation accepts a maximum of ten entries per call and the deployment monitors more regions than that.

Flood Forecast Lambda Folder

The forecast handler returns a list of failed message identifiers rather than throwing, which is the mechanism that makes partial batch failure effective.

Figure 8

Flood Forecast Handler Source Showing Partial Batch Failure Reporting

SCREENSHOT INSTRUCTION: Capture the forecast handler source code.
Capture: Code editor, infra/lambdas/fg-flood-forecast/index.mjs.
Command: grep -n "batchItemFailures" -A 4 -B 8 infra/lambdas/fg-flood-forecast/index.mjs
Expected Evidence: The handler collecting itemIdentifier values into batchItemFailures and returning it.
Placement: Immediately after the paragraph above.

Report Intake Lambda Folder

The intake handler validates its request body before any write, since it accepts unauthenticated input and must reject malformed submissions with a client error rather than a server error.

Alert Dispatch Lambda Folder

The dispatch handler publishes to SNS only after the alert record is created, so a publication never precedes a durable record.

Shared Helper Files

A single shared module provides structured logging, environment variable resolution, and the authenticated internal fetch helper used by every function that writes back to the application server.

Lambda Microservices

Install Lambda Dependencies

Only the presign function requires third-party dependencies, namely the S3 client and request presigner packages. The remaining functions rely on the runtime provided SDK, which keeps their deployment packages small and their cold start latency low.

Package Lambda Code

Each function is packaged as a compressed archive containing its handler and a copy of the shared module. Table 3 records the resulting configuration of the six deployed functions.

Table 3
Deployed Configuration of the Six Lambda Microservices

Function	Trigger	Timeout (s)	Memory (MB)	Tracing
fg-weather-ingest	EventBridge, 10 min	300	512	Active
fg-flood-forecast	SQS, batch 5	120	512	Active
fg-alert-dispatch	SQS, batch 1	60	512	Active
fg-report-intake	API Gateway	30	512	Active
fg-upload-presign	API Gateway	30	512	Active
fg-image-process	S3 object created	30	512	Active

Note. All functions use the Node.js 22 runtime on the arm64 architecture. Timeout values reflect the longest observed execution for each workload plus headroom.

Create Weather Ingestion Lambda

The ingestion function is created with a three hundred second timeout, reflecting its ten sequential outbound API calls, and with active tracing enabled so that those calls appear as subsegments in X-Ray.

Figure 9

CLI Resource Provisioning and Verification of the Lambda Functions

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the Lambda functions.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 44
Verification Command: aws lambda list-functions --query "Functions[?starts_with(FunctionName,'fg-')].{Name:FunctionName,Runtime:Runtime,Tracing:TracingConfig.Mode}" --output table
Expected Evidence: Six functions, runtime nodejs22.x, tracing Active.
Placement: Immediately after the paragraph above.

Create Flood Forecast Lambda

The forecast function is invoked from the queue rather than from the network, so it requires no invoke permission for API Gateway. Its event source mapping is described in Section *Event Source Configuration*.

Create Report Intake Lambda

The intake function is the public write path and is deployed with the narrowest execution role in the system.

Create Alert Dispatch Lambda

The dispatch function holds the only SNS publish permission in the serverless tier, which confines the ability to notify residents to a single component.

Event Source Configuration

Configure EventBridge Schedule

An EventBridge Scheduler schedule invokes the ingestion function every ten minutes. This replaces the in-process cron and, critically, fires exactly once regardless of how many application instances are running.

Figure 10

CLI Resource Provisioning and Verification of the EventBridge Schedule

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the schedule.
CLI: AWS CLI v2

Provisioning Command: cd infra && ./deploy.sh 46
Verification Command: aws scheduler get-schedule --name fg-weather-ingest-schedule --query '{State:State,Expression:ScheduleExpression,Target:Target.Arn}'
Expected Evidence: Output shows state ENABLED, the expression rate(10 minutes), and the ingestion function as target.
Placement: Immediately after the paragraph above.

Configure SQS Event Source Mappings

Two event source mappings connect the work queues to their consumers. Both enable partial batch failure reporting, and both specify a maximum concurrency. The concurrency ceiling is essential rather than cosmetic: this account permits ten concurrent function executions, and an uncapped queue consumer will scale until it owns all of them, starving the endpoints on the public request path. The rationale and the measured consequence are examined in Section *CloudWatch Monitoring Evidence*.

Figure 11
Verification of the SQS Event Source Mappings and Concurrency Ceilings
SCREENSHOT INSTRUCTION: Capture both event source mappings with their concurrency limits.
Capture: Terminal, AWS CLI.
Command: aws lambda list-event-source-mappings --query 'EventSourceMappings[].{Function:FunctionArn,Batch:BatchSize,MaxConcurrency:ScalingConfig.MaximumConcurrency,State:State}' --output table
Expected Evidence: Two enabled mappings, with maximum concurrency 3 for forecast and 2 for dispatch.
Placement: Immediately after the paragraph above.

API Gateway REST API

Create REST API

A REST API named fg-serverless-api fronts the two network-addressable functions. The REST protocol was selected over the newer HTTP API protocol for one decisive reason: HTTP APIs do not emit X-Ray trace segments, and tracing evidence is a requirement of this task (Amazon Web Services, 2025c). The cost difference is immaterial at this request volume.

Figure 12

CLI Resource Provisioning and Verification of the API Gateway REST API

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the REST API.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 45
Verification Command: aws apigateway get-rest-apis --query "items[?name=='fg-serverless-api'].{Id:id,Name:name}" --output table
Expected Evidence: The API identifier and name.
Placement: Immediately after the paragraph above.

Create API Stage

A single stage named v1 was deployed. Stage names appear in the request path, so the invoke URL terminates in /v1.

Enable X-Ray Tracing

Active tracing was enabled on the stage. This produces an API Gateway stage node in the service map, which is what allowed a fault to be attributed to the gateway rather than to a handler during performance analysis.

API Gateway Integrations and Lambda Invoke Permissions

Create Report Intake Integration

The report resource uses Lambda proxy integration, which forwards the entire request to the handler and returns the handler response verbatim. This keeps request shaping in application code rather than in gateway mapping templates.

Create Upload Presign Integration

The presign resource is configured identically, targeting the presign function.

Allow API Gateway to Invoke Report Intake Lambda

A resource-based policy statement permits the gateway to invoke the function. Without it the gateway receives an authorisation failure and returns a server error, a symptom easily mistaken for a defect in the handler.

Allow API Gateway to Invoke Upload Presign Lambda

The equivalent statement was applied to the presign function.

Create API Gateway Routes

Report Routes

The report resource exposes a POST method for submission and an OPTIONS method so that browser preflight requests succeed.

Upload Routes

The presign resource is configured equivalently.

Deployed Routes

A consolidated listing confirms the complete resource tree as deployed.

Figure 13

Complete Deployed Resource Tree of the REST API

SCREENSHOT INSTRUCTION: Capture every deployed resource path and its methods.
Capture: Terminal, AWS CLI.
Command: aws apigateway get-resources --rest-api-id 60h0a79hgb --query 'items[].{Path:path,Methods:resourceMethods}' --output json
Expected Evidence: Output shows the /sl, /sl/reports, /sl/uploads, and /sl/uploads/presign paths, with methods present on the two leaf resources.
Placement: Immediately after the paragraph above.

Frontend Environment and Build

Create Environment File

The frontend requires the API base path at build time, because framework environment variables prefixed for public exposure are inlined into the browser bundle during compilation rather than read at runtime. The relative path /api is used, which means no rebuild is required if the distribution domain changes.

Build the Frontend

The frontend is compiled to a standalone output bundle. It must be built with npm rather than pnpm: standalone output traces dependencies into a generated module directory, and the symlinked store used by pnpm omits transitive-only packages from that trace, producing a runtime module resolution failure that surfaces as an opaque gateway error.

Deploy the Frontend

The compiled bundle is deployed to the Elastic Beanstalk web environment. The deployment step asserts that the previously described module resolves before shipping the artefact, converting a class of runtime failure into a build-time failure.

Figure 14

CLI Deployment and Health Verification of the Web Environment

SCREENSHOT INSTRUCTION: Capture the terminal immediately after deploying the frontend environment.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 31
Verification Command: aws elasticbeanstalk describe-environments --query 'Environments[?Status!=Terminated].{Env:EnvironmentName,Health:Health,Status:Status}' --output table
Expected Evidence: Both environments report health Green, status Ready.
Placement: Immediately after the paragraph above.

CloudFront Distribution

Create Cache and Origin Request Policies

A single distribution serves three origins. Static framework assets are cached immutably, while API and serverless paths forward the full request without caching, since responses are request-specific.

Create CloudFront Distribution

Consolidating the three Task 1 distributions into one removes the cross-origin configuration that separate hostnames required. Every browser request is now same-origin, so no preflight occurs anywhere in the system.

Figure 15

CLI Resource Provisioning and Verification of the CloudFront Distribution

SCREENSHOT INSTRUCTION: Capture the terminal after provisioning the distribution.
CLI: AWS CLI v2
Provisioning Command: cd infra && ./deploy.sh 32
Verification Command: aws cloudfront get-distribution --id E7JA2N5C3YF8P --query 'Distribution.{Status:Status,Domain:DomainName,Enabled:DistributionConfig.Enabled}'
Expected Evidence: Output shows status Deployed, the distribution domain name, and enabled true.
Placement: Immediately after the paragraph above.

Add the Serverless Path Behaviour

A final behaviour routes the serverless path prefix to the API Gateway origin, completing the single front door. Table 4 records the resulting routing table.

Table 4

CloudFront Path Routing to the Three Origins

Path pattern	Origin	Cached	Data owned by origin
/_next/static/*	Elastic Beanstalk web	Yes	None
/ (default)	Elastic Beanstalk web	No	None
/api/*	Elastic Beanstalk application	No	PostgreSQL
/sl/*	API Gateway	No	DynamoDB and S3

Note. Because all four patterns resolve to one hostname, the browser treats every request as same-origin, which removes cross-origin preflight from the request path entirely.

Figure 16

End-to-End Verification of All Three Origin Routes

SCREENSHOT INSTRUCTION: Capture all three origins responding correctly through the single distribution hostname.
Capture: Terminal, curl.
Command: curl -s -o /dev/null -w "%{http_code}\n" https://drtyovliurlkl.cloudfront.net/ then repeat for /api/health and /sl/uploads/presign (POST)
Expected Evidence: All three return HTTP 200, so all three origins are reachable through one hostname.
Placement: Immediately after the paragraph above.

CloudWatch Monitoring Evidence

Monitoring was implemented as a fourteen-widget dashboard laid out along the request path, together with twenty-one alarms, every one of which publishes to the operations topic. Metrics without alarm actions constitute observation without response, so no metric was instrumented without a corresponding threshold. All handlers emit single-line structured logs, which makes fields queryable in CloudWatch Logs Insights.

Three alarms carry disproportionate diagnostic weight. A non-empty dead-letter queue indicates a message that has failed three delivery attempts and would otherwise represent silent data loss. The age of the oldest queued message is a more honest backlog signal than queue depth, because depth appears healthy on a large but fast-moving queue even while consumers fall behind. Function throttling indicates concurrency exhaustion, which is a scaling signal rather than a code defect.

The throttling alarm proved its value during load testing. An initial run returned two server errors out of two hundred and fifty-three gateway requests. The traces recorded thirteen millisecond durations with an empty function segment, which indicates rejection before application code executed rather than handler failure. The throttling metric confirmed two rejections, and the account quota query revealed the cause: ten concurrent executions rather than the default one thousand. Because neither queue consumer had a concurrency ceiling at that point, a queue backlog could have consumed the entire pool and starved the public request path. Applying the ceilings described in Section *Configure SQS Event Source Mappings* eliminated the condition, as Table 5 records.

Table 5

Throttling and Server Error Counts Before and After Concurrency Allocation

Measurement run	Presign invocations	Throttled invocations	Gateway server errors
Before allocation,	118	2	2

Measurement run	Presign invocations	Throttled invocations	Gateway server errors
burst of 12			
After allocation, burst of 8	117	0	0

Note. Counts were obtained at sixty-second resolution from the AWS/Lambda and AWS/ApiGateway namespaces. Functional verification passed in both runs; only the metrics distinguished a working system from one able to withstand a queue backlog.

Table 6 summarises the performance measured across a thirty-minute window covering the load test.

Table 6

Measured Performance of the Serverless Tier

Function	Invocations	Errors	Mean (ms)	95th percentile (ms)
fg-upload-presign	245	0	13.1	96.2
fg-report-intake	4	0	277.7	551.0
fg-flood-forecast	12	0	466.9	968.6
fg-alert-dispatch	4	0	475.4	765.9
fg-image-process	2	0	557.5	772.2
fg-weather-ingest	5	0	1,638.3	2,194.8

Note. API Gateway recorded 253 requests with a mean latency of 59.6 ms, a 95th percentile of 144.4 ms, and mean integration latency of 56.8 ms. The divergence between mean and 95th percentile latency reflects initialisation of new execution environments during burst traffic rather than processing cost.

The ingestion function is the slowest at 1.6 seconds, which is expected because it performs ten sequential outbound API calls. That figure is the clearest justification for moving scoring behind a queue: a synchronous design would have placed those seconds on a user request. Across the same window both work queues recorded equal counts of messages sent, received, and deleted, and both dead-letter queues remained empty, indicating no message loss and no redrive.

Figure 17

CloudWatch Dashboard Showing Request Path Metrics Under Load

SCREENSHOT INSTRUCTION: Capture the monitoring dashboard populated with load test data.

Capture: Console, CloudWatch, Dashboards, FloodGuard-Task2, time range set to one hour.

Command: Run `cd infra && ./deploy.sh 63` first so the widgets contain traffic, then open the dashboard.

Expected Evidence: Gateway, function, and queue widgets all show plotted data rather than empty axes. Capture the upper and lower halves separately so axis labels stay readable.

Placement: Immediately after the paragraph above.

Figure 18

CloudWatch Alarm Inventory in a Non-Alarming State

SCREENSHOT INSTRUCTION: Capture all project alarms and their current state.

Capture: Console, CloudWatch, All alarms, filtered by the prefix `fg-`.

Command: `aws cloudwatch describe-alarms --query "MetricAlarms[?starts_with(AlarmName,'fg-')].{Name:AlarmName,State:StateValue}" --output table`

Expected Evidence: Twenty-one alarms, all in state OK.

Placement: Immediately after the paragraph above.

Figure 19

AWS X-Ray Service Map Spanning the Server and Serverless Tiers

SCREENSHOT INSTRUCTION: Capture the X-Ray service map covering both tiers.

Capture: Console, CloudWatch, X-Ray traces, Service map, time range set to one hour.

Command: Run `cd infra && ./deploy.sh 63` first to generate traffic, then open the service map.

Expected Evidence: The gateway stage node, the function nodes, and the application server node connected in one map.

Placement: Immediately after the paragraph above.

Figure 20

X-Ray Trace Timeline Showing Function Initialisation Cost

SCREENSHOT INSTRUCTION: Capture a single trace timeline that includes an initialisation subsegment.

Capture: Console, CloudWatch, X-Ray traces, Traces list, opening a `POST /sl/uploads/presign` trace.

Command: `aws xray get-trace-summaries --start-time $(date -u -d '15 minutes ago' +%s) --end-time $(date -u +%s) --query 'TraceSummaries[0].Id' --output text`

Expected Evidence: The gateway segment and function subsegments with individual durations, showing initialisation latency.

Placement: Immediately after the paragraph above.

Figure 21

Consolidated End-to-End Verification Result

SCREENSHOT INSTRUCTION: Capture the automated verification summary table.

Capture: Terminal, project root.

Command: `cd infra && ./deploy.sh 63`

Expected Evidence: Summary shows eighteen checks passed, zero failed, covering routing, authorisation, validation, uploads, ingestion, dead-letter queues, deduplication, and traces.

Placement: Immediately after the paragraph above.

Results

This section presents the functional outcome of the implementation from the perspective of each user role the platform serves. All screens were captured from the live deployment reached through the single CloudFront hostname. Where a capability is exercised by the serverless tier rather than by a page, the evidence is presented at the interface level instead.

Public Users

Unauthenticated visitors can reach public information and submit a flood report without registering, which matters during an emergency when account creation is an unreasonable precondition to raising an alarm.

Homepage

The landing page presents current regional risk levels and entry points to authentication.

Figure 22
Public Landing Page Served Through the CloudFront Distribution
SCREENSHOT INSTRUCTION: Capture the public landing page as served to an unauthenticated visitor.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/.
Command: Not applicable (browser view).
Expected Evidence: Rendered page with the distribution hostname in the address bar, confirming delivery via CloudFront.
Placement: Immediately after the paragraph above.

Alerts Page

The alerts view lists active flood alerts for the regions relevant to the signed-in resident, ordered by severity.

Figure 23
Resident Alerts View Listing Active Flood Alerts
SCREENSHOT INSTRUCTION: Capture the alerts listing.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/resident/alerts.
Command: Not applicable (browser view).
Expected Evidence: At least one pipeline-generated alert, with severity and affected region.
Placement: Immediately after the paragraph above.

Map Page

The map view renders monitored regions coloured by the risk level written by the forecast function, which makes the output of the serverless pipeline directly visible to residents.

Figure 24
Interactive Risk Map Reflecting Computed Regional Risk Levels
SCREENSHOT INSTRUCTION: Capture the interactive risk map.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/resident/map.
Command: Not applicable (browser view).
Expected Evidence: Regions must be visibly colour-coded by risk level, with a legend present.
Placement: Immediately after the paragraph above.

Guest Report Submission

Guest reporting is served by the serverless tier rather than by the application server. The intake function accepts an unauthenticated report through API Gateway and rejects a malformed submission with a client error, which is the behaviour a public endpoint must exhibit.

Figure 25
Unauthenticated Guest Report Accepted and Malformed Report Rejected
SCREENSHOT INSTRUCTION: Capture both a successful guest report submission and the rejection of an invalid submission.
Capture: Terminal, curl.
Command: curl -s -o /dev/null -w "invalid body: %{http_code}\n" -X POST https://drtyovliurlkl.cloudfront.net/sl/reports -H 'content-type: application/json' -d '{"description":"x"}'
Expected Evidence: The malformed body returns HTTP 400 and a valid one returns HTTP 201, so validation discriminates.
Placement: Immediately after the paragraph above.

Create Account

Registration collects the minimum necessary detail and assigns the resident role by default. Role elevation is an administrative action rather than a self-service one.

Figure 26
Account Registration Form
SCREENSHOT INSTRUCTION: Capture the completed registration form immediately before submission.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/register.
Command: Not applicable (browser view).

Expected Evidence: Completed form with the password masked. Use a test account, not a personal credential.
Placement: Immediately after the paragraph above.

Credential Verification

Authentication uses signed tokens issued by the application server, with the signing secret drawn from Secrets Manager. Verification is therefore demonstrated by showing that a valid credential yields a token while an invalid one is refused, rather than by an electronic mail confirmation code.

Sign In

Successful authentication redirects each user to the dashboard appropriate to their assigned role.

Figure 27
Authentication Sign-In Page
SCREENSHOT INSTRUCTION: Capture the sign-in page with the test account electronic mail entered.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/login.
Command: Not applicable (browser view).
Expected Evidence: The sign-in form must be visible with the password field masked.
Placement: Immediately after the paragraph above.

Citizen Dashboard

The resident dashboard aggregates local risk level, active alerts, and the resident's own submissions into a single view.

Figure 28
Resident Dashboard Following Successful Authentication
SCREENSHOT INSTRUCTION: Capture the authenticated resident dashboard.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/resident.
Command: Not applicable (browser view).
Expected Evidence: Region risk level and active alerts, confirming role-based routing after sign-in.
Placement: Immediately after the paragraph above.

Submit Report

Report submission with a photograph exercises the serverless upload path end to end. The browser requests a presigned URL from the presign function, transmits the image directly to S3, and the resulting object notification invokes the verification function. This is the clearest single demonstration of the serverless integration because no image byte passes through the application server.

Figure 29

Report Submission Showing the Direct Browser-to-S3 Upload Sequence

SCREENSHOT INSTRUCTION: Capture the report submission form with the browser network inspector open alongside it.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/resident/reports, with developer tools open on the Network tab.
Command: Not applicable (browser view).
Expected Evidence: POST /sl/uploads/presign followed by a separate PUT to the S3 hostname, proving the image bypassed the application server. Truncate the signed URL query string; it is a temporary credential.
Placement: Immediately after the paragraph above.

Figure 30

Object Tags Written by the Image Verification Function

SCREENSHOT INSTRUCTION: Capture the verification tags applied to the uploaded object.
Capture: Console, S3, floodguard-uploads-292960609118, reports/ prefix, the newly uploaded object, Properties, Tags.
Command: aws s3api get-object-tagging --bucket floodguard-uploads-292960609118 --key reports/<OBJECT-KEY>
Expected Evidence: Tags verification=verified and detectedType are present, proving the object notification invoked the function and the signature check passed.
Placement: Immediately after the paragraph above.

Authority Dashboard

The authority dashboard presents incoming reports awaiting triage together with regional status, giving an operator a single queue of work.

Figure 31

Authority Dashboard Showing Reports Awaiting Triage

SCREENSHOT INSTRUCTION: Capture the authority dashboard.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/admin, signed in with an administrative account.

Command: Not applicable (browser view).
Expected Evidence: Pending reports and regional status summaries must be visible.
Placement: Immediately after the paragraph above.

Verify or Reject Report

An operator confirms or dismisses a submitted report. Only verified reports contribute to regional assessment, which prevents a malicious or mistaken submission from influencing risk.

Figure 32
Report Verification Decision Recorded by an Authority User
SCREENSHOT INSTRUCTION: Capture a report transitioning to the verified state.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/admin/reports, with a single report opened.
Command: Not applicable (browser view).
Expected Evidence: Status changed to verified, with the photograph rendered because the function tagged it a genuine image.
Placement: Immediately after the paragraph above.

Dispatch Rescuers

Verified reports requiring intervention are converted into assistance requests that volunteers can claim. Claiming is exclusive, so two volunteers cannot be dispatched to the same task.

Figure 33
Volunteer Assistance Request Claimed for Dispatch
SCREENSHOT INSTRUCTION: Capture an assistance request assigned to a volunteer.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/volunteer/requests, signed in with a volunteer account.
Command: Not applicable (browser view).
Expected Evidence: An assigned volunteer and claimed status, demonstrating exclusive assignment.
Placement: Immediately after the paragraph above.

Update Report Status to Resolved

Closing the loop, an operator marks the incident resolved once the assistance request is complete, which removes it from the active queue while retaining it for analysis.

Issue Alerts

An authority user may issue an alert manually in addition to those generated automatically by the forecast pipeline. Both paths converge on the same dispatch function, so both benefit from the duplicate suppression window.

Figure 34

Manual Alert Issued by an Authority User

SCREENSHOT INSTRUCTION: Capture the alert creation interface immediately after issuing an alert.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/admin/alerts.
Command: Not applicable (browser view).
Expected Evidence: The newly created alert must appear in the active alert list with its severity and target region.
Placement: Immediately after the paragraph above.

Alert Notification Reaches Users

Delivery is the outcome that matters most in an early warning system. SNS fan-out places the alert in subscriber inboxes, and the resident interface reflects the same alert, confirming both channels operate from one authoritative record.

Figure 35

Alert Notification Delivered to a Subscriber by Amazon SNS

SCREENSHOT INSTRUCTION: Capture the delivered notification received by a subscribed test address.
Capture: Electronic mail client inbox showing the received flood alert notification.
Command: aws cloudwatch get-metric-statistics --namespace AWS/SNS --metric-name NumberOfMessagesPublished --dimensions Name=TopicName,Value=fg-flood-alerts --period 3600 --statistics Sum --start-time <ISO8601> --end-time <ISO8601>
Expected Evidence: Message shows alert severity and region. Obscure the recipient address.
Placement: Immediately after the paragraph above.

Admin Dashboard

The administrative analytics view aggregates alert volume, severity distribution, and the most frequently affected regions, supporting review after an event rather than response during one.

Figure 36

Administrative Analytics View

SCREENSHOT INSTRUCTION: Capture the analytics dashboard.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/admin/analytics.
Command: Not applicable (browser view).
Expected Evidence: Key indicator tiles and at least one populated chart must be visible.
Placement: Immediately after the paragraph above.

Manage Users

User administration permits role assignment across the resident, volunteer, and authority roles. This is the only path by which a user gains elevated permission.

Figure 37

User Administration and Role Assignment

SCREENSHOT INSTRUCTION: Capture the user management interface.
Capture: Browser, https://drtyovliurlkl.cloudfront.net/dashboard/admin/users.
Command: Not applicable (browser view).
Expected Evidence: The user list with assigned roles. Obscure real electronic mail addresses.
Placement: Immediately after the paragraph above.

User Profile Management

Each user maintains their own contact details and notification preferences, which determine the channels through which SNS reaches them.

Figure 38

User Profile and Notification Preference Management

SCREENSHOT INSTRUCTION: Capture the user profile management view.
Capture: Web browser, signed in as the test resident, at the profile section of the resident dashboard.
Command: Not applicable (browser view).
Expected Evidence: Editable contact details and notification preferences. Use the test account only.
Placement: Immediately after the paragraph above.

References

Adzic, G., & Chatley, R. (2017). Serverless computing: Economic and architectural impact. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (pp. 884–889). Association for Computing Machinery. https://doi.org/10.1145/3106237.3117767

Amazon Web Services. (2025a). *Lambda function scaling*. AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html

Amazon Web Services. (2025b). *Using Lambda with Amazon SQS*. AWS Lambda Developer Guide. https://docs.aws.amazon.com/lambda/latest/dg/with-sqs.html

Amazon Web Services. (2025c). *Tracing user requests to REST APIs using X-Ray*. Amazon API Gateway Developer Guide. https://docs.aws.amazon.com/apigateway/latest/developerguide/apigateway-xray.html

Amazon Web Services. (2025d). *Using Amazon CloudWatch alarms*. Amazon CloudWatch User Guide. https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/AlarmThatSendsEmail.html

Amazon Web Services. (2025e). *Configuring event notifications using Amazon S3 event notifications*. Amazon S3 User Guide. https://docs.aws.amazon.com/AmazonS3/latest/userguide/NotificationHowTo.html

Amazon Web Services. (2025f). *Amazon SQS dead-letter queues*. Amazon SQS Developer Guide. https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

Amazon Web Services. (2025g). *Reliability pillar: AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/reliability-pillar/welcome.html

Baldini, I., Castro, P., Chang, K., Cheng, P., Fink, S., Ishakian, V., Mitchell, N., Muthusamy, V., Rabbah, R., Slominski, A., & Suter, P. (2017). Serverless computing: Current trends and open problems. In S. Chaudhary, G. Somani, & R. Buyya (Eds.), *Research advances in cloud computing* (pp. 1–20). Springer. https://doi.org/10.1007/978-981-10-5026-8_1

Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N., Gonzalez, J. E., Popa, R. A., Stoica, I., & Patterson, D. A. (2019). *Cloud programming simplified: A Berkeley view on serverless computing* (Technical Report No. UCB/EECS-2019-3). University of California, Berkeley. https://doi.org/10.48550/arXiv.1902.03383

Newman, S. (2021). *Building microservices: Designing fine-grained systems* (2nd ed.). O'Reilly Media.

Open-Meteo. (2025). *Weather forecast API documentation*. https://open-meteo.com/en/docs

Richardson, C. (2018). *Microservices patterns: With examples in Java*. Manning Publications.

Sbarski, P., & Kroonenburg, S. (2017). *Serverless architectures on AWS: With examples using AWS Lambda*. Manning Publications.